Q35. Case Study: Financial Data Processing System — Merge Sort vs Quick Sort

1. Aim

To implement and compare **Merge Sort** and **Quick Sort** for sorting financial transaction data and analyze their **stability, execution speed, memory usage, time complexity, and suitability** using the divide-and-conquer approach.

---

2. Problem Statement

A financial data processing system stores transaction records containing transaction IDs and transaction amounts. These transactions need to be sorted according to their amount for reporting and analysis.

The system requires an efficient sorting algorithm. Therefore, **Merge Sort** and **Quick Sort** are implemented and compared based on:

- Sorting performance
- Stability
- Memory usage
- Best, average, and worst-case complexity
- Divide-and-conquer principles
- Suitability for financial transaction processing

---

3. Divide-and-Conquer Principle

Both Merge Sort and Quick Sort are based on the **divide-and-conquer strategy**.

Merge Sort

Merge Sort works in three steps:

1. **Divide:** Divide the transaction array into two approximately equal halves.
2. **Conquer:** Recursively sort each half.
3. **Combine:** Merge the two sorted halves into one sorted array.

For example:

[5000, 1200, 5000, 750]

↓

Divide into

[5000,1200] [5000,750]

↓

Recursively sort

[1200,5000] [750,5000]

↓

Merge

↓

[750,1200,5000,5000]

Quick Sort

Quick Sort works as follows:

1. **Divide:** Select a pivot element.
2. **Partition:** Place elements smaller than the pivot on one side and larger elements on the other.
3. **Conquer:** Recursively sort both partitions.

Combine: No separate merging is required because the elements are already partitioned around the pivot.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define N 100000


typedef struct {
    int id;
    int amount;
    int originalOrder;
} Transaction;

// ------------------------------------------------
// MERGE SORT
// ------------------------------------------------

void merge(Transaction arr[], int left, int mid, int right) {
```

```
    int n1 = mid - left + 1;
    int n2 = right - mid;

    Transaction *L = (Transaction *)malloc(n1 * sizeof(Transaction));
    Transaction *R = (Transaction *)malloc(n2 * sizeof(Transaction));

    int i, j, k;

    for (i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for (j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    i = 0;
    j = 0;
    k = left;



        if (L[i].amount <= R[j].amount)
            arr[k++] = L[i++];

        else
            arr[k++] = R[j++];
    }

    while (i < n1)
        arr[k++] = L[i++];

    while (j < n2)
        arr[k++] = R[j++];

    free(L);
    free(R);
}

void mergeSort(Transaction arr[], int left, int right) {


  if (left < right) {

      int mid = left + (right - left) / 2;

      // Divide
      mergeSort(arr, left, mid);
      mergeSort(arr, mid + 1, right);

      // Combine
      merge(arr, left, mid, right);
  }
}

// ------------------------------------------------
// QUICK SORT
// ------------------------------------------------
```

```
    for (int j = low; j < high; j++) {
        if (arr[j].amount < pivot) {
            i++;
            Transaction temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    Transaction temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

printf("Quick Sort:\n");
printf("Best Case    : O(n log n)\n");
printf("Average Case : O(n log n)\n");
printf("Worst Case   : O(n^2)\n");
printf("Space        : O(log n) average\n");
printf("Stable       : No\n\n");

printf("================================================\n");
printf(" RECOMMENDATION\n");
printf("================================================\n");

printf("Merge Sort is recommended for the financial\n");
printf("transaction system because it is stable and\n");
printf("provides guaranteed O(n log n) performance.\n");

// Free memory
free(data);
free(mergeData);
```

```
 FINANCIAL DATA PROCESSING SYSTEM
 MERGE SORT VS QUICK SORT
==========================================

ORIGINAL TRANSACTIONS
ID      Amount  Original Order
101     5000    0
102     1200    1
103     5000    2
104     750     3
105     1200    4
106     3000    5
107     750     6
108     9000    7

AFTER MERGE SORT
ID      Amount  Original Order
104     750     3
107     750     6
102     1200    1
105     1200    4
106     3000    5
101     5000    0
103     5000    2
108     9000    7

AFTER QUICK SORT
ID      Amount  Original Order
107     750     6
104     750     3
105     1200    4
```

7. Stability Analysis

Stability means that when two transactions have the same sorting value, their original relative order is preserved.

For example, transactions 101 and 103 both have an amount of ₹5000.

Original:

101 → ₹5000

103 → ₹5000

After Merge Sort:

101 → ₹5000

103 → ₹5000

Their order is preserved.

Therefore, **Merge Sort is stable**.

In the Quick Sort implementation, partitioning can exchange equal-valued elements. Therefore, the original order is not guaranteed.

Therefore, **Quick Sort is not stable**.

Stability can be important in financial applications because transactions with equal amounts may need to maintain their original timestamp, transaction ID, or input order for reporting and auditing.

---

8. Performance Analysis

Both algorithms generally provide good performance.

Merge Sort

Best Case = O(n log n)

Average Case = O(n log n)

Worst Case = O(n log n)

Merge Sort guarantees O(n log n) performance even in the worst case.

Quick Sort

Best Case = O(n log n)

Average Case = $O(n \log n)$

Worst Case = $O(n^2)$

Quick Sort is usually very fast in practical situations, but poor pivot selection can result in the worst-case $O(n^2)$ performance.

For the sample execution with 100,000 transactions:

Merge Sort : 15.906 ms

Quick Sort : 5.449 ms

Thus, **Quick Sort was faster in this particular execution**.

However, execution time alone should not determine the algorithm selection because financial systems may also require stability and predictable worst-case performance.

---

9. Memory Usage Analysis

Merge Sort

Merge Sort requires an additional temporary array during the merge operation.

Extra Space = $O(n)$

Therefore, Merge Sort uses more memory.

Quick Sort

The implementation performs partitioning in-place.

Its average recursion-stack requirement is:

$O(\log n)$

Therefore, Quick Sort generally uses less additional memory.

Feature	**Merge Sort**	**Quick Sort**
Technique	Divide and Conquer	Divide and Conquer
Best Case	$O(n \log n)$	$O(n \log n)$
Average Case	$O(n \log n)$	$O(n \log n)$
Worst Case	**$O(n \log n)$**	**$O(n^2)$**

Feature	Merge Sort	Quick Sort
Extra Space	**O(n)**	**O(log n) average**
Stable	**Yes**	**No**
In-place	No	Yes
Practical Speed	Good	Usually very good
Predictable Performance	**Yes**	No
Suitable for equal-value ordering	**Yes**	No guarantee

11. Advantages and Disadvantages

Merge Sort — Advantages

1. Stable sorting algorithm.
2. Guaranteed O(n log n) worst-case performance.
3. Suitable for large datasets.
4. Predictable execution time.
5. Useful when the relative order of equal transactions must be maintained.

Merge Sort — Disadvantages

1. Requires O(n) additional memory.
2. Requires extra copying during merging.
3. May be slower than Quick Sort in practical in-memory situations.

Quick Sort — Advantages

1. Usually very fast in practice.
2. Requires less additional memory.
3. Works in-place in the given implementation.
4. Good cache performance.
5. Average-case complexity is O(n log n).

Quick Sort — Disadvantages

1. Basic Quick Sort is not stable.

2. Worst-case complexity can become $O(n^2)$.
3. Performance depends on pivot selection.
4. Equal transaction values may not preserve their original order.

---

12. Financial System Analysis

For a financial transaction system, sorting is not only about speed.

Suppose the system has:

Transaction 101 → ₹5000

Transaction 103 → ₹5000

If both transactions have the same amount, their original order may represent their transaction sequence.

A stable algorithm preserves:

101 → 103

instead of potentially changing it to:

103 → 101

This can be useful for:

- Financial reports
- Transaction histories
- Audit trails
- Secondary sorting
- Maintaining chronological ordering
- Consistent reporting

Therefore, **stability is an important factor in this case study**.

---

13. Which Algorithm Is More Suitable?

Merge Sort is the recommended algorithm.

Although Quick Sort may be faster and use less additional memory, Merge Sort provides two major advantages for financial processing:

1. **Stable sorting**
2. **Guaranteed O(n log n) worst-case performance**

Financial systems often prioritize **correctness, predictable performance, and audit consistency** over a small improvement in average execution time.

Therefore:

Merge Sort is the most suitable solution for the given financial transaction processing system when stability and predictable performance are important.

Quick Sort can still be selected when:

- Stability is not required.
- Memory usage is a major concern.
- Average-case speed is the primary requirement.
- A robust pivot strategy is implemented.

---

14. Final Conclusion

Merge Sort and Quick Sort both use the **divide-and-conquer technique**, but they differ in their implementation, memory requirements, stability, and worst-case performance.

Merge Sort divides the data into smaller subarrays, recursively sorts them, and merges them. It provides O(n log n) performance in all cases and is stable, but requires O(n) additional memory.

Quick Sort selects a pivot and partitions the data around it. It is generally faster in practical situations and uses less additional memory, but it is not stable and can have $O(n^2)$ worst-case performance.

For the financial transaction processing system, **Merge Sort is recommended** because maintaining transaction order and ensuring predictable performance are important requirements.
